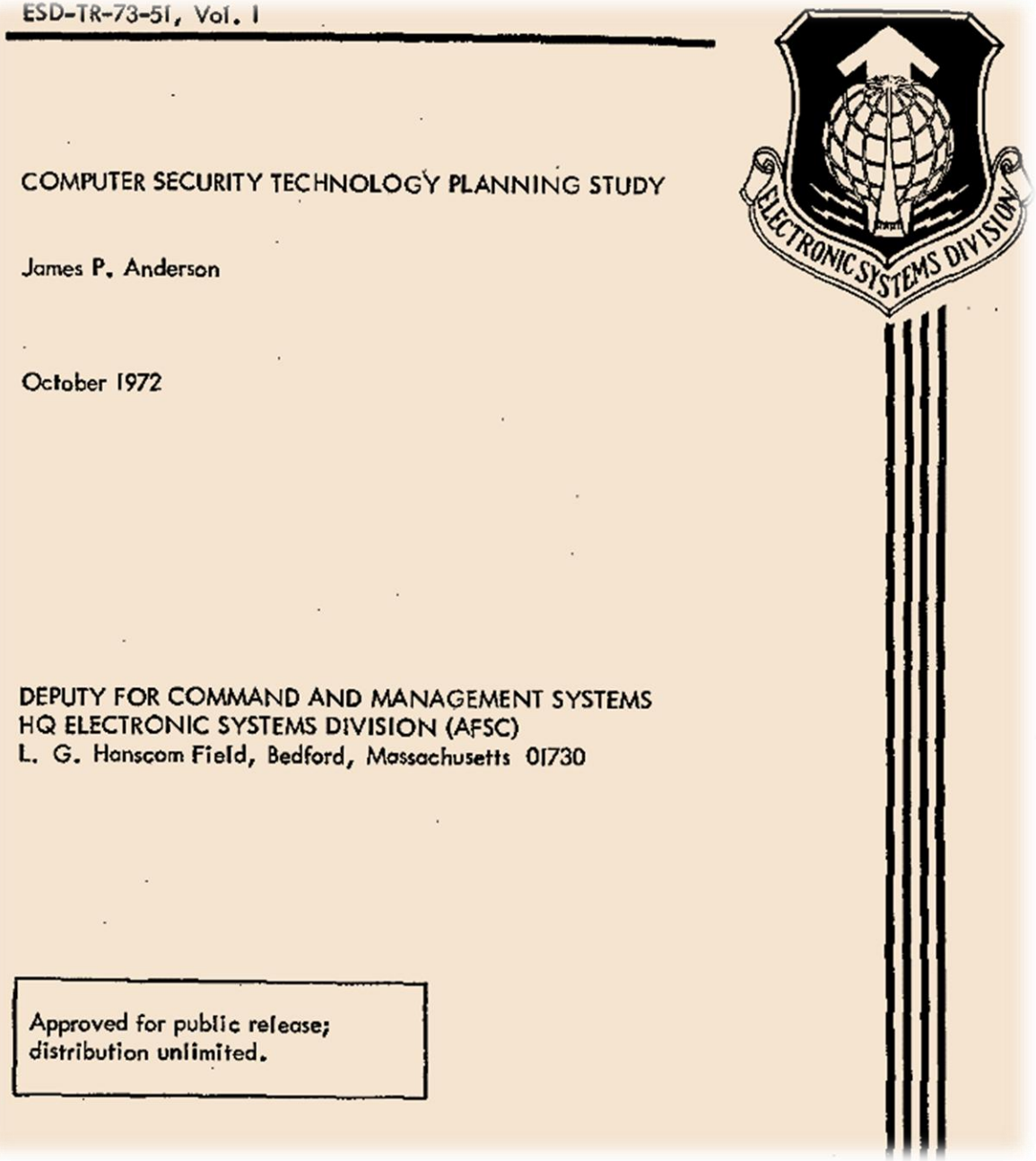
بسمه تعالی



تشخیص نفوذ
درس دوم : ناظر مرجع و مبنای رایانشی معتمد


ESD-TR-73-51, Vol. I

COMPUTER SECURITY TECHNOLOGY PLANNING STUDY

James P. Anderson

October 1972

DEPUTY FOR COMMAND AND MANAGEMENT SYSTEMS
HQ ELECTRONIC SYSTEMS DIVISION (AFSC)
L. G. Hanscom Field, Bedford, Massachusetts 01730

Approved for public release;
distribution unlimited.

ELECTRONIC SYSTEMS DIVISION

مساله قدیمی

- در سال ۱۹۷۲ گزارشی در نتیجه سفارش نیروی هوایی آمریکا منتشر شد که حاصل کارگروهی صنعتی و دانشگاهی و ... بود.
- صورت مساله: تهدید ناشی از کاربر مخرب در سیستم های open use که به دنبال بهره برداری از نقص های طراحی یا پیاده سازی سیستم است و نیاز به تدوین استانداردهای جهانی امنیتی برای آن
- تحلیل به دست آمده مبنای معیارهای tcsec شد که استاندارد ارزیابی امنیتی سیستم وزارت دفاع آمریکاست و در حوزه امنیت بسیار تاثیرگذار بود.

The technical issue of multilevel computer security is concerned with the concept of malicious threat. By this we recognize that the nature of shared use multilevel computer systems present to a malicious user a unique opportunity for attempting to subvert through programming the mechanism upon which security depends (i. e., the control of the computer vested in the operating system). This threat, coupled with the concentration of the application (data, control system, etc.) in one place (the computer system) makes computers a uniquely attractive target for malicious (hostile) action. Recognition of the implication of malicious threat is important to understanding the security limitations surrounding application of contemporary computer systems. The threat that a single user of a system operating as a hostile agent can simply modify an operating system to by-pass or suspend security controls, and the fact that the operating system controlling the computer application(s) is developed outside of USAF control, contribute strongly to the reluctance to certify (i. e., be convinced) that contemporary systems are secure or even can be secured.

- در سیستم های کامپیوتری چندسطحی مشترک (shared multi-level) مهاجم این فرصت را دارد تا ساز و کار امنیتی (مثل تشخیص نفوذ) را دور بزند.
- همچنین، همه چیز در یک سیستم متمرکز است.
- پس یک کاربر می تواند سیستم عامل را طوری تغییر دهد که کنترل های امنیتی را معلق کند یا دور بزند و سیستم معتمد نماند.

دور خوردن قلاب های نظارتی

- Of all the components included in modern endpoint security products, the most
- widely deployed are DLLs responsible for function hooking, or interception. These DLLs provide
- defenders with a large amount of important information related to code execution, such as the
- parameters passed to a function of interest and the values it returns.
- hijacking function calls with strategic rewrites of jump instructions.
- `detours`, a popular library, replaces code snippets with `detours` (custom functions) and trampolines (original code containers) to achieve this, allowing developers to intercept and manipulate function behavior

دور زدن تکنیک های نظارتی XDR

- قلاب های نظارتی: دور زدن قلاب ها
- استفاده از ویندوز درایورها
- استفاده از ETW ویندوز:
  - تغییر پیکربندی
  - وصله در حافظه
- تکنیک های Living off the Land (LOTL) : سو استفاده از ابزارهای معتمد موجود و پرواز زیر چشم رادار!

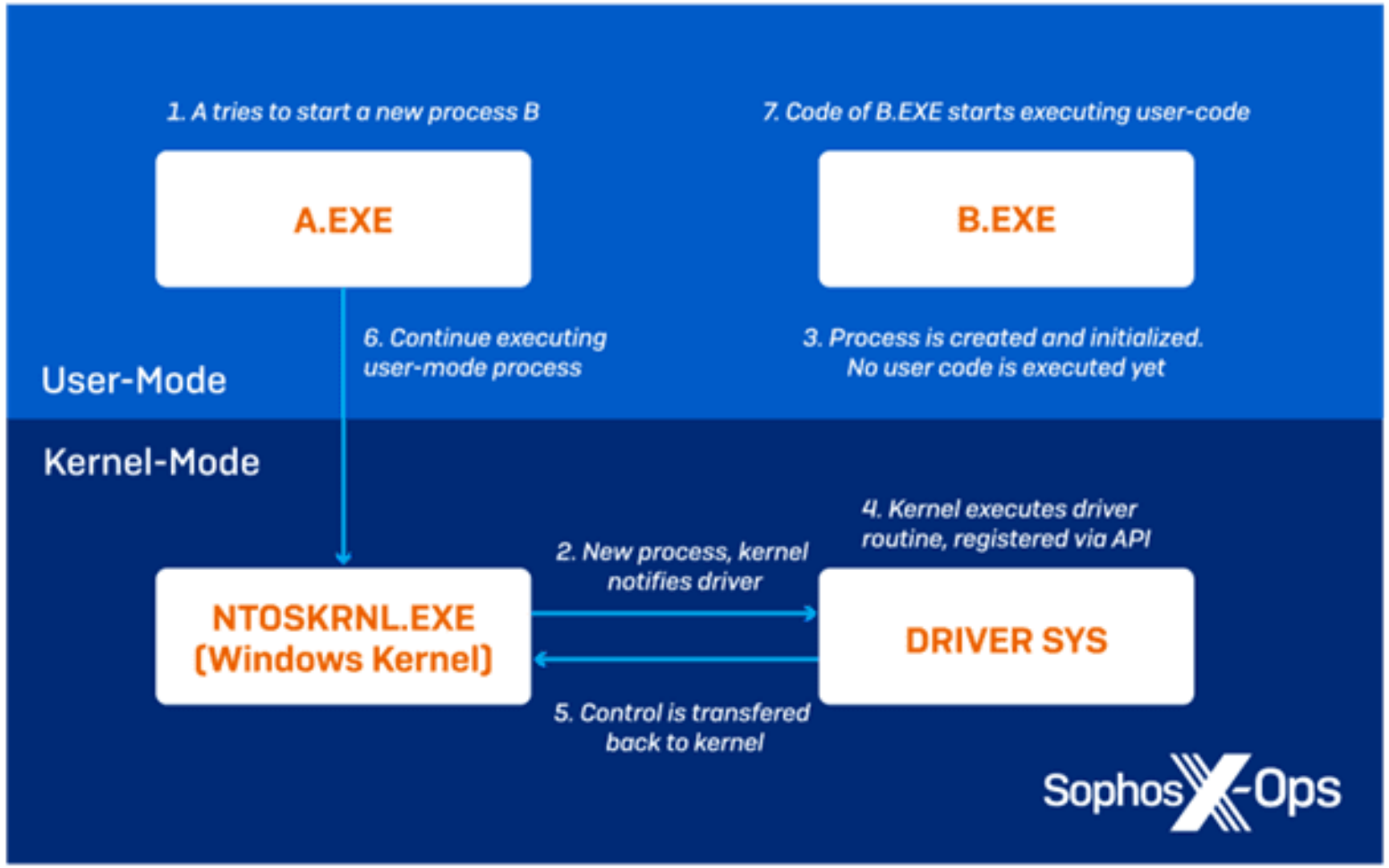
چطور CPU بگیریم؟

Table 15.1 Ways to Capture CPU Time

Strategy	Tactic	Elements Altered
Modify static elements	Hooking	IAT, SSDT, GDT, IDT, MSRs
	In-place patching	System calls, driver routines
	Detour patching	System calls, driver routines
Modify dynamic elements	Alter repositories	Registry hives, event logs
	DKOM	EPROCESS, DRIVER_SECTION structures
	Alter callbacks	PE image `.data` section

Go to a spe

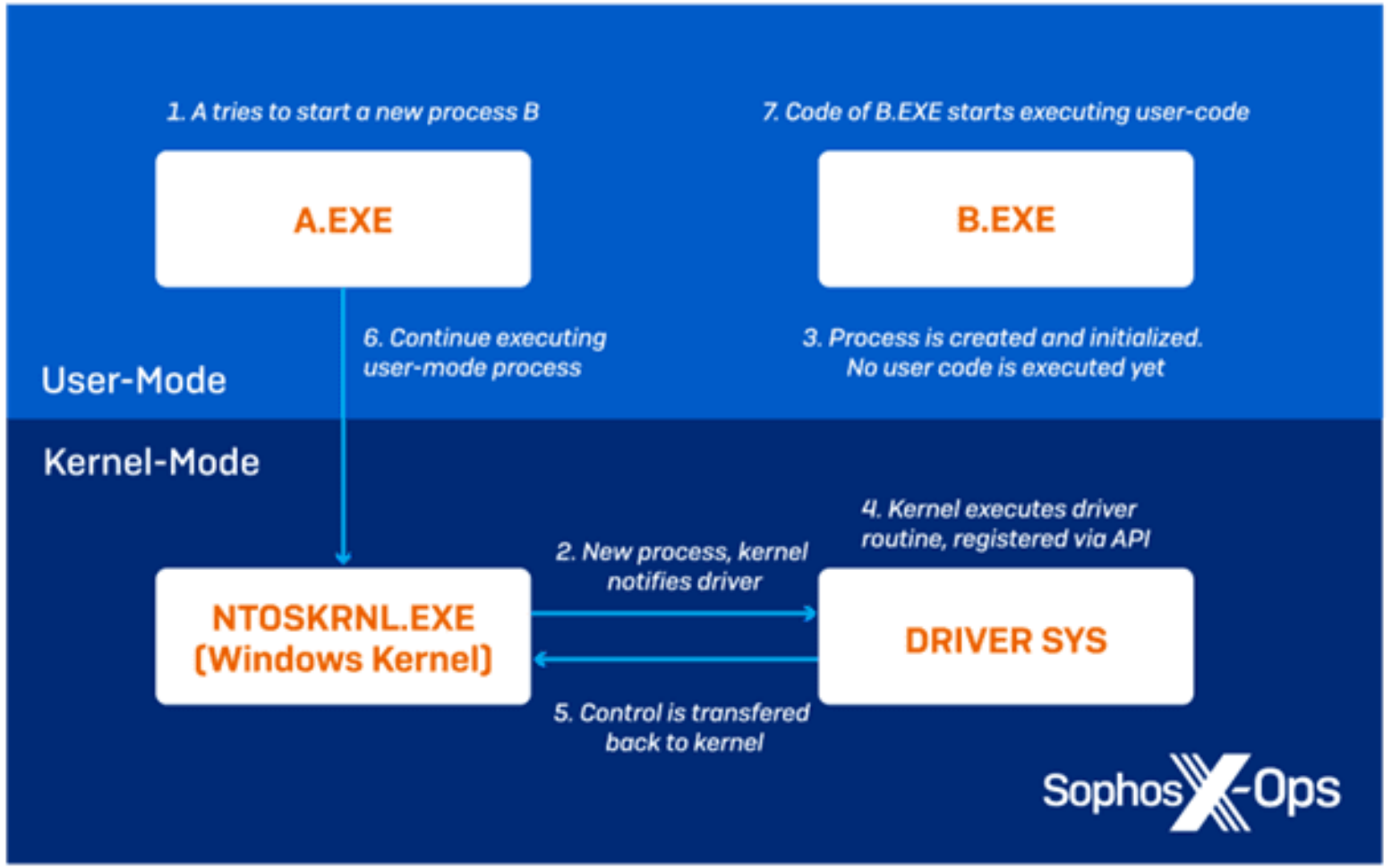
یک نمونه درایور هسته اوست چگونه کار می کند؟



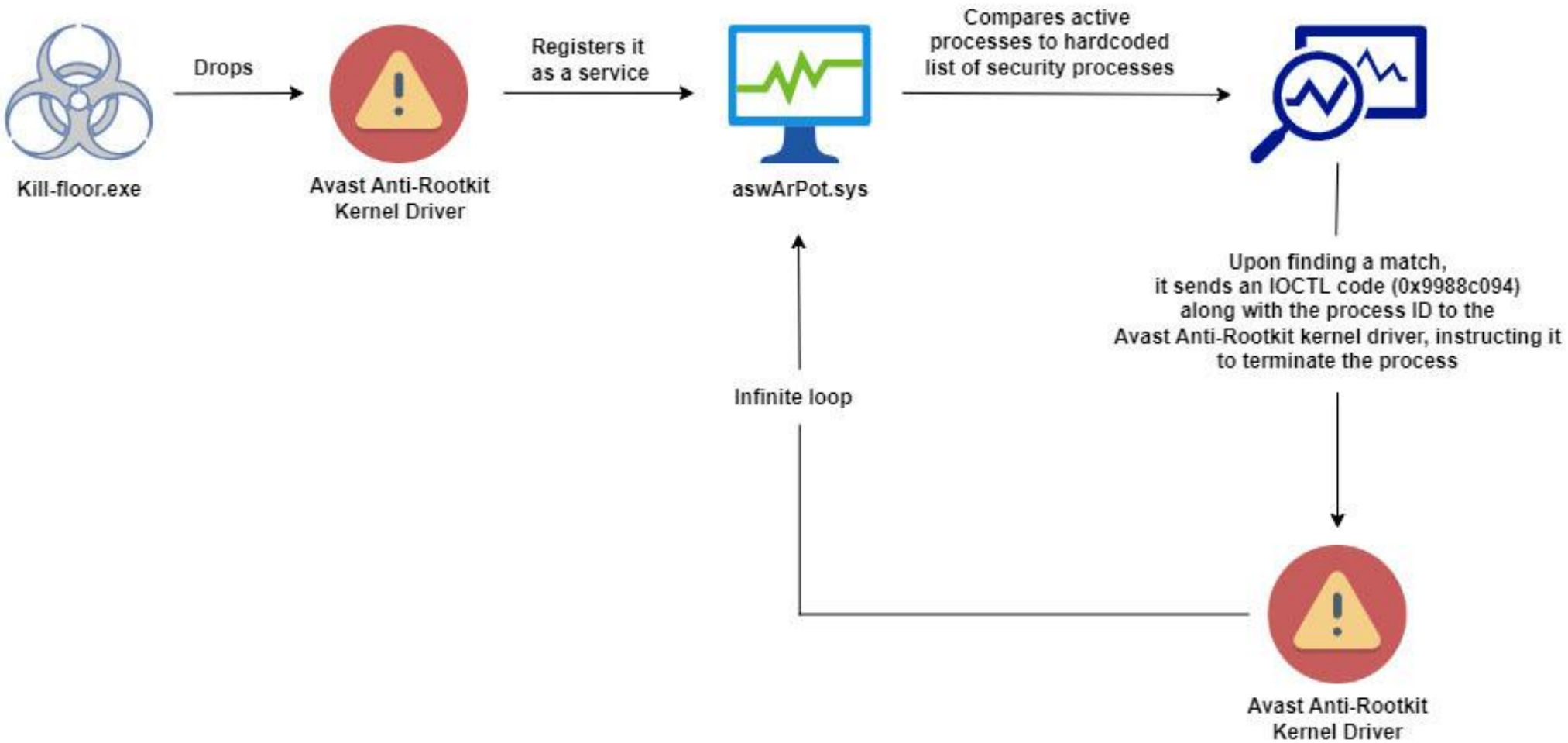
یک نمونه درایور هسته اوست چگونه کار می کند؟



قابل اعتماد
شد ؟



مثال اخیر: بدافزارkill-floor (۲۰۲۵)



درایورهای آسیب پذیر، به سلاح تبدیل شده، و خصمانه

Vulnerable Driver: gdrv.sys

`gdrv.sys` is a kernel driver developed by Gigabyte. It enables various motherboard enhancements, including hardware management, overclocking and performance optimization.

Older versions of this driver contain multiple vulnerabilities — such as CVE-2018-19320, CVE-2018-19321, CVE-2018-19322 and CVE-2018-19323 — that enable an attacker to perform arbitrary reads and writes to kernel memory, IO ports and MSR registers. One example of its abuse was RobbinHood ransomware, which exploited a vulnerability exposed via the `0xC3502808` input/output control code (IOCTL)[1] to disable DSE with the purpose of loading its own malicious driver. This malicious driver would then be used to remove security software such as EDR tools.

```
switch ( ioctl )
{
  case 0xC3502804:
    status = sub_127C0(DeviceExtension, Irp, CurrentStackLocation);
    break;
  case 0xC3502808:
    status = f_custom_memcpy(DeviceExtension, Irp);
    break;
  case 0xC350280C:
    status = sub_128F0(DeviceExtension, Irp);
```

Weaponizable Driver: GMER

GMER is a tool designed to help identify rootkits hidden in the operating system kernel. It enables auditing and tampering with various kernel structures to achieve its purpose of identifying and protecting against rootkits. This powerful tool requires a driver to achieve all of this functionally — a driver that has been weaponized by various actors and open-source tooling to terminate or suspend protected processes.

This driver exposes the `ZwTerminateProcess` kernel API via the `0x9876C094` IOCTL. The user mode process needs to send only the process ID (PID) of the process to be terminated.

```
case 0x9876C094:
  if ( a3_inputLen != 4 || !a2_IRP_buffer )
  {
    *a7 = STATUS_INVALID_BUFFER_SIZE;
    return STATUS_IN_PAGE_ERROR|0x200LL;
  }
  v44 = f_WrapTerminateProcess(*a2_IRP_buffer);
```

Figure 3. IOCTL 0x9876C094 receives the PID of the target process to terminate

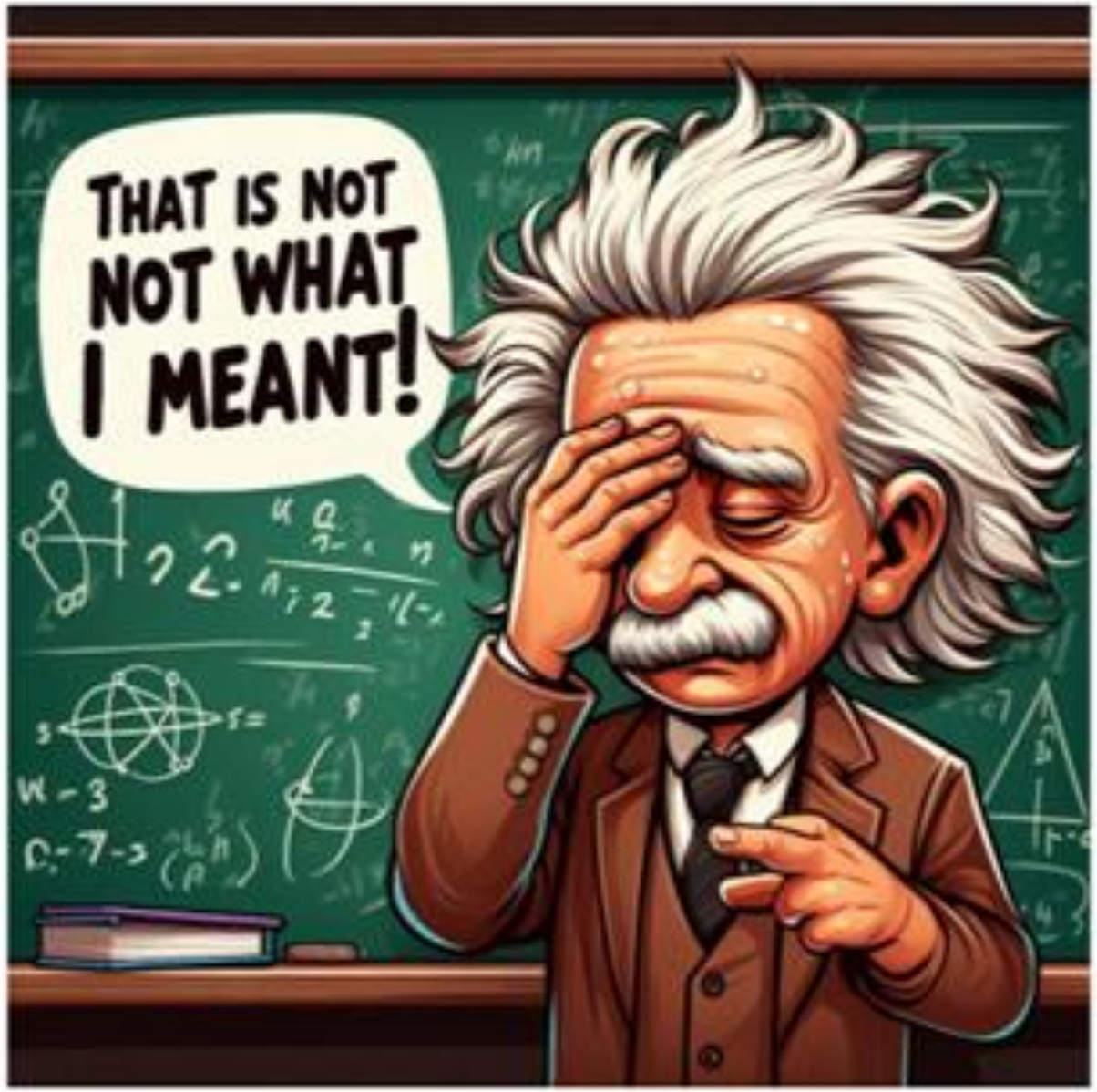
Malicious Driver: Mimikatz

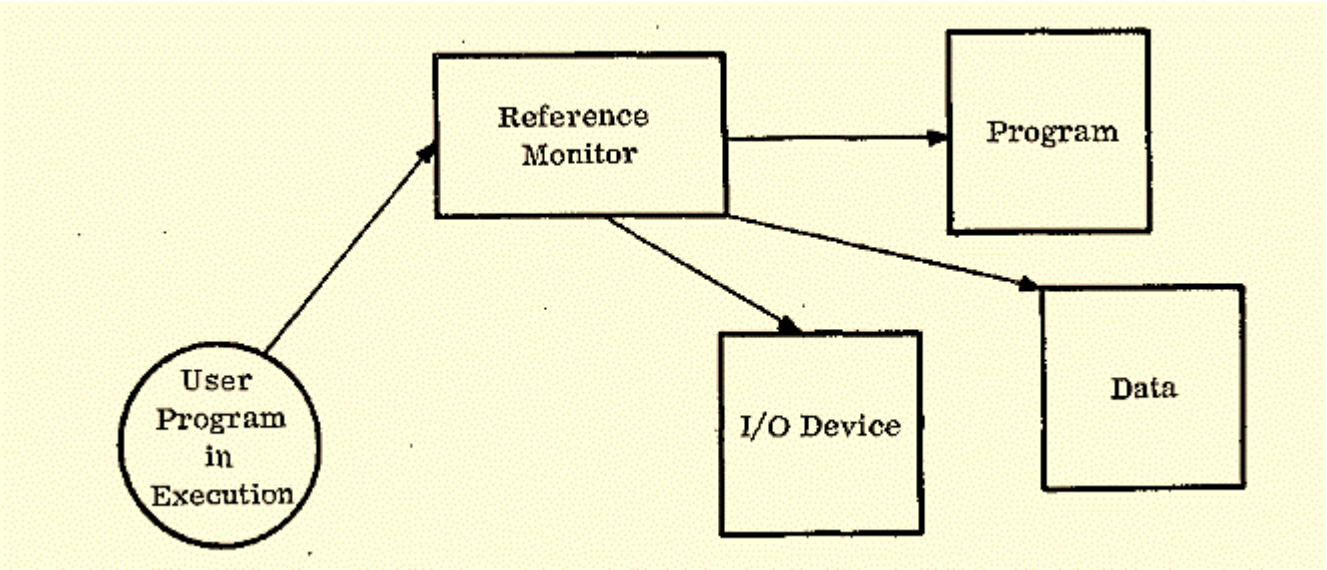
Mimikatz[2] is a well-known tool used by security professionals as well as adversaries to extract Windows OS credentials. Once credentials are harvested, these can later be used to achieve lateral movement within the network and compromise other endpoints.

Mimikatz also has a kernel driver that is signed with a legitimate certificate, allowing it to load without disabling Windows DSE. Due to its functionalities, the Mimikatz driver is considered to be malicious software, despite it being written by a security professional.

راه حل چیست ؟

مفهومی تئوری به نام ناظر مرجع (Reference monitor) با این هدف تعریف شد.



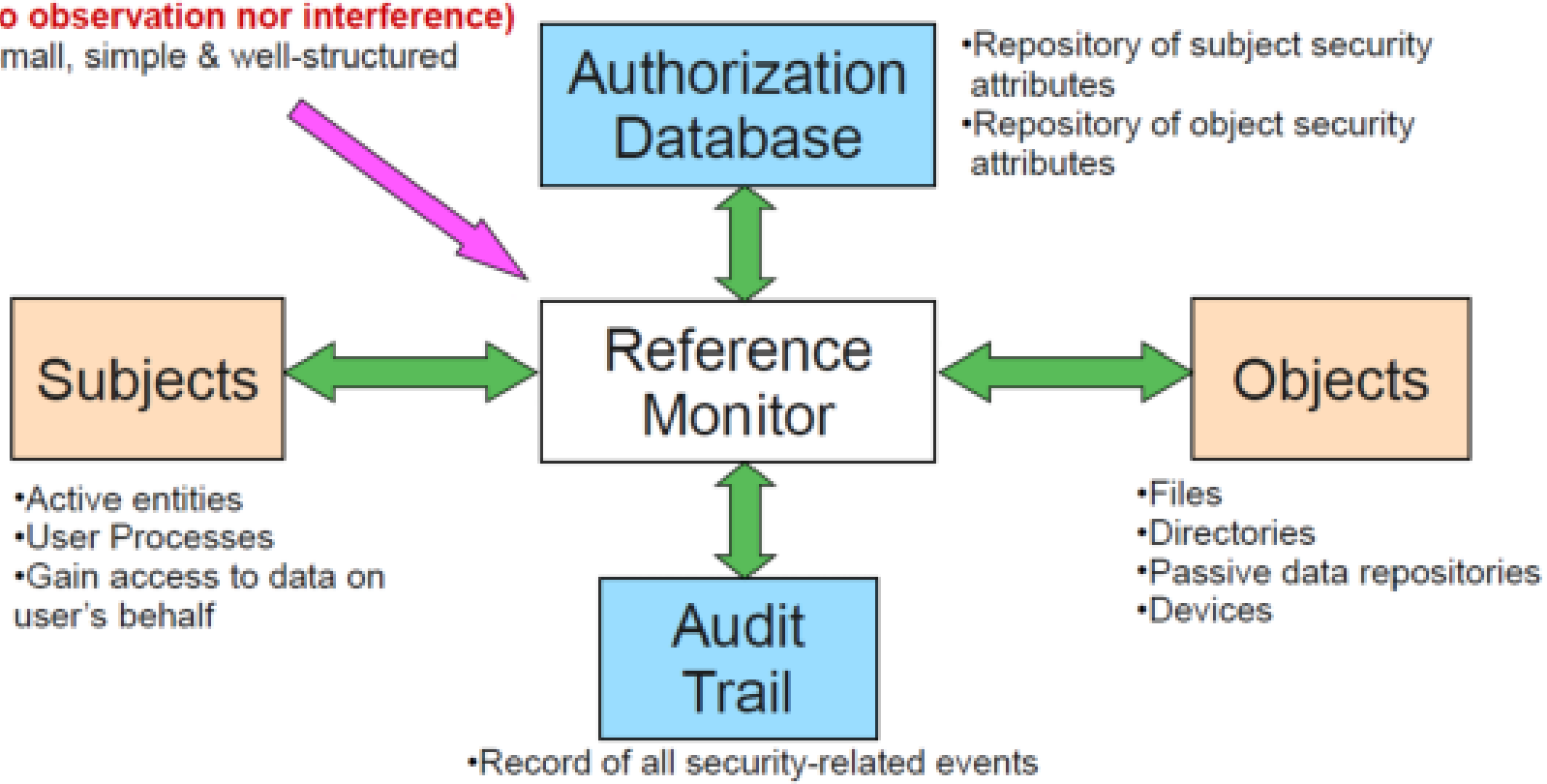


One of the most promising developments of this idea is the concept of a reference monitor[2] which enforces the authorized access relationships between subjects and objects of a system. This is illustrated in Figure 1. An implementation of the reference monitor concept is a reference validation mechanism[3] that validates each reference to data or programs by any user (program) against a list of authorized types of reference for that user. The authorized access relationships between subjects and objects are defined in terms of privilege (e. g. READ or WRITE applied to data objects and EXECUTE applied to program objects). As a means of depicting these relationships, it is convenient to use a matrix, with subjects making access listed opposite rows and objects to which access is made listed above the columns. The entries in the matrix define for each subject whether access is permitted to the object and with what privileges. An example of such a matrix is shown in Figure 2.

The Reference Monitor

(A Secure System Architecture USAF, October 1972)

- Enforces security policy
- Mediates all attempts by subjects to access objects
- Tamperproof database & audit trail **(no observation nor interference)**
- Small, simple & well-structured



- رصدگر مرجع موضوعی در حوزه نقطه اعمال سیاست است.
- پس رصدگر نه کشف آنومالی می کند و نه از داده چیزی می آموزد.

نیازمندی های رصدگر مرجع

- اعتبارسنجی انجام شده توسط ناظر باید این ویژگی ها را داشته باشند:
  - مکانیسم اعتبارسنجی مرجع باید **قابل تأیید** (verfiaible) - باید به اندازه کافی کوچک باشد تا بتوان آن را به طور رسمی (فرمال) تجزیه و تحلیل و تأیید کرد.
  - مکانیسم اعتبارسنجی مرجع باید **همیشه فراخوانی شود** (always-invoked). بدون این ویژگی، ممکن است مکانیسم در زمان مورد نظر اجرا نشود و به مهاجم اجازه دهد سیاست امنیتی را نقض کند.
  - مکانیسم اعتبارسنجی مرجع باید **ضد دستکاری** (tamper-proof) باشد. بدون این ویژگی، یک مهاجم می‌تواند خودِ سازوکار را تضعیف کند و از این رو سیاست امنیتی را نقض کند.

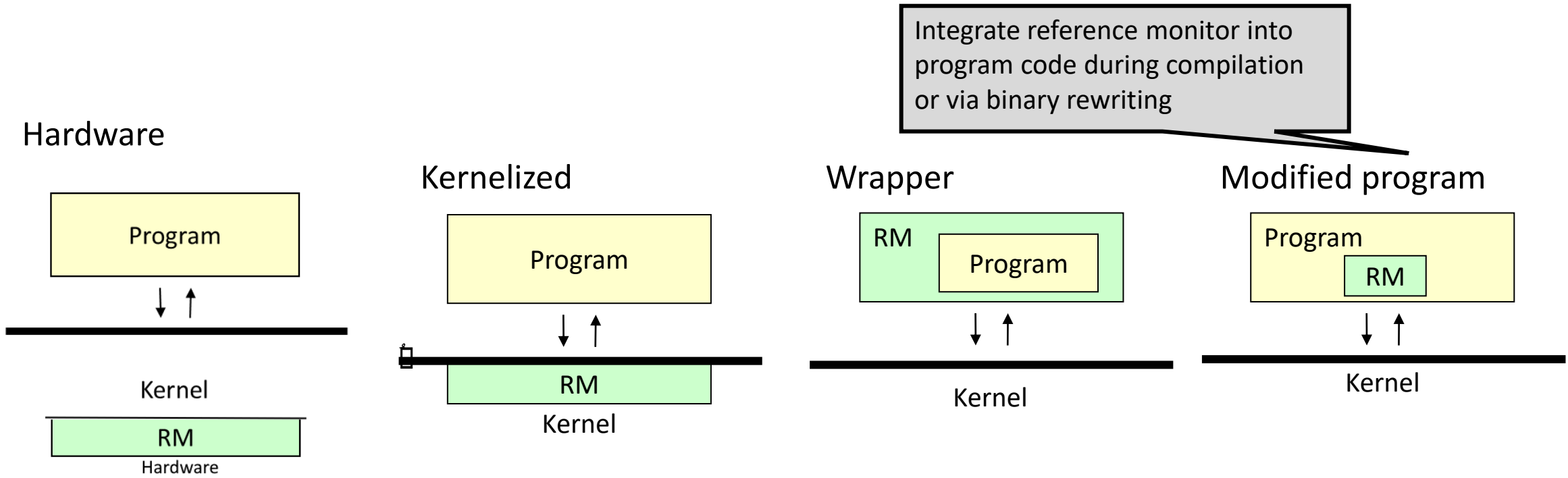
ارتباط ناظر مرجع با IDS چیست ؟

- در زمینه IDS، یک ناظر مرجع می‌تواند:
- تمام فراخوانی‌های سیستم یا ارتباطات بین فرآیندی را رصد کند.
- سیاست‌های امنیتی را اعمال کند (مثلاً رفتارهای خاص را مجاز/منع کند).
- به عنوان یک پایه قابل اعتماد عمل کند که منطق IDS بر اساس آن عمل می‌کند.
- بحثی ذیل اعتماد به خود IDS، یعنی:
  - چقدر IDS قادر است رفتارهای سیستم تحت نظارت خود را کامل و جامع ببیند؟
  - چقدر IDS قابل دور زدن، یا دریافت اطلاعات جعلی است ؟

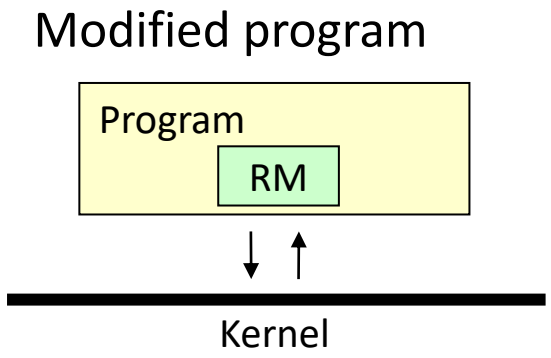
دامنه عملکرد ناظر مرجع

- اجرای برنامه/پردازه ها را مشاهده می‌کند
  - در چه سطحی؟ سخت‌افزار، سیستم‌عامل، شبکه
- اگر برنامه در شرف نقض سیاست امنیتی باشد، اجرا را متوقف یا محدود می‌کند.
  - «سیاست امنیتی» چیست؟
  - کدام رویدادهای سیستمی به این سیاست مربوط می‌شوند؟
    - دستورالعمل‌ها، دسترسی به حافظه، فراخوانی‌های سیستمی، بسته‌های شبکه...
- توسط فرآیند تحت نظارت قابل دور زدن نیستند.
- بیشتر مکانیسم‌های اجرایی که خواهیم دید، نمونه‌هایی از مانیتورهای مرجع هستند.

انواع پیاده سازی ناظر مرجع



- ✓ پایین‌ترین سطح / پیاده‌سازی سخت‌افزاری: ناظر مرجع مستقیماً در سطح سخت‌افزار یا ریزکد پیاده‌سازی می‌شود (مانند معماری‌های قدیمی یا پردازنده‌های امن).
- ✓ هسته‌ای: ناظر مرجع مستقیماً در داخل هسته سیستم‌عامل تعبیه شده است (که به ایده‌های اولیه اندرسون و هسته‌های امن نزدیک‌تر است).
- ✓ لفاف : ناظر مرجع مانند یک لایه یا پوشش (مثل مانیتورهای زمان اجرا یا کانتینرها/سندباکس‌ها) برنامه را احاطه می‌کند و درخواست‌های آن را رهگیری می‌کند بدون اینکه حتماً جزو هسته باشد.
- ✓ برنامه تغییر یافته: کدهای بررسی امنیت مستقیماً در حین کامپایل یا بازنویسی باینری (Binary Rewriting)درون خودِ کد برنامه تزریق می‌شوند.



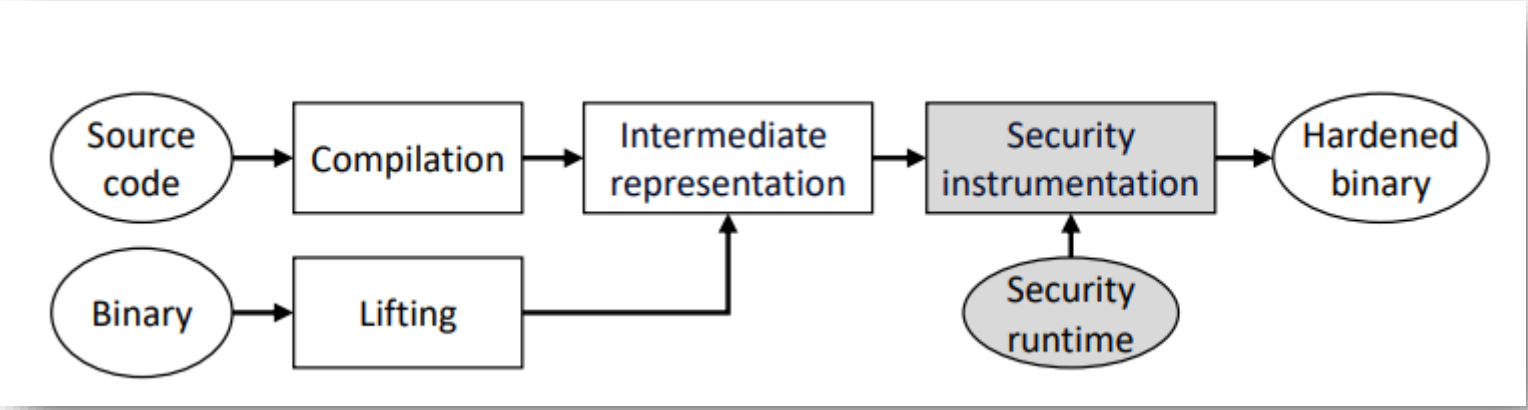
نمونه ای از مفاهیم ذیل ناظر درون برنامه هدف

Inlined Reference Monitor, IRM

رصدگر مرجع در-خط

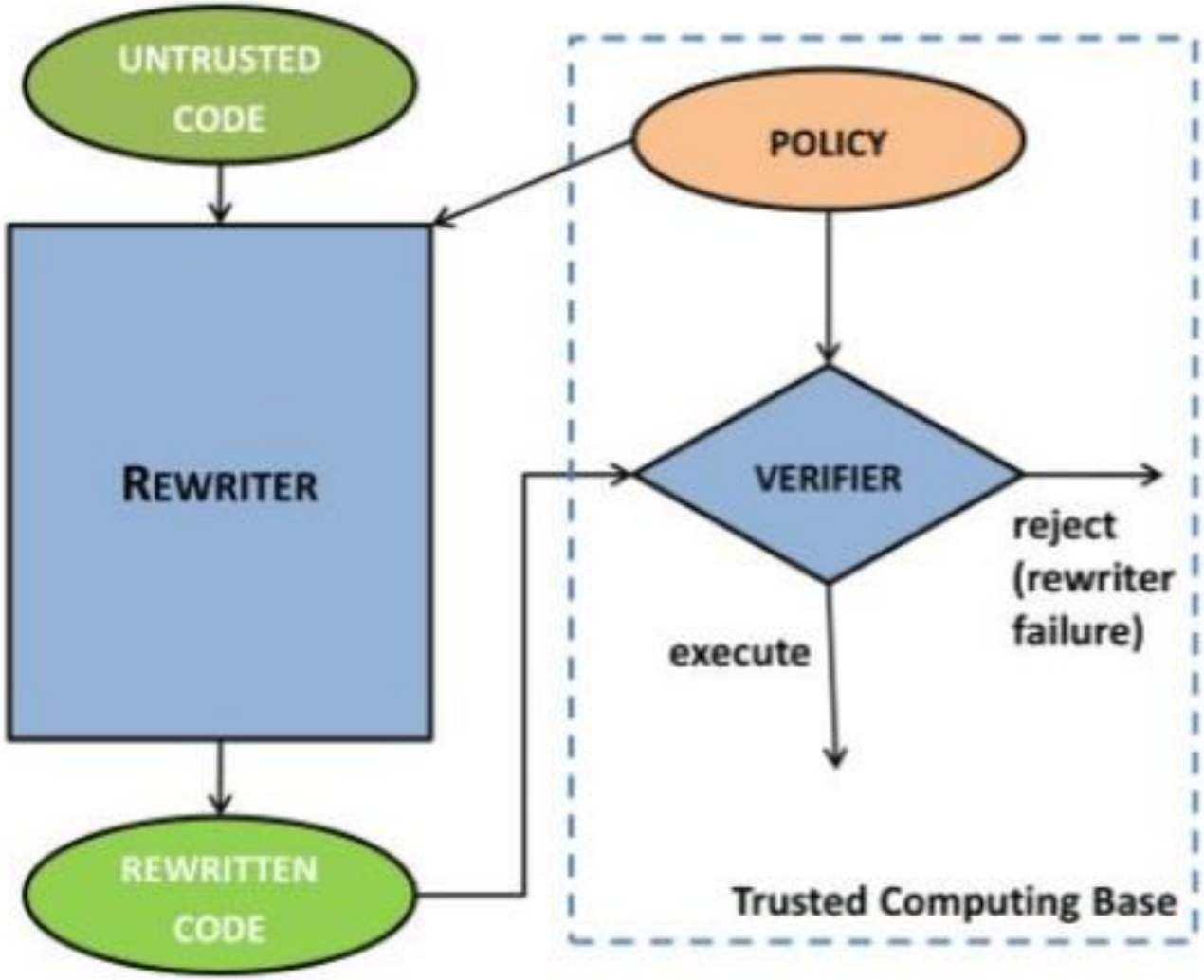
از ۱۹۹۶ طرح شده و هنوز مفهوم کاربردی و زنده ای است

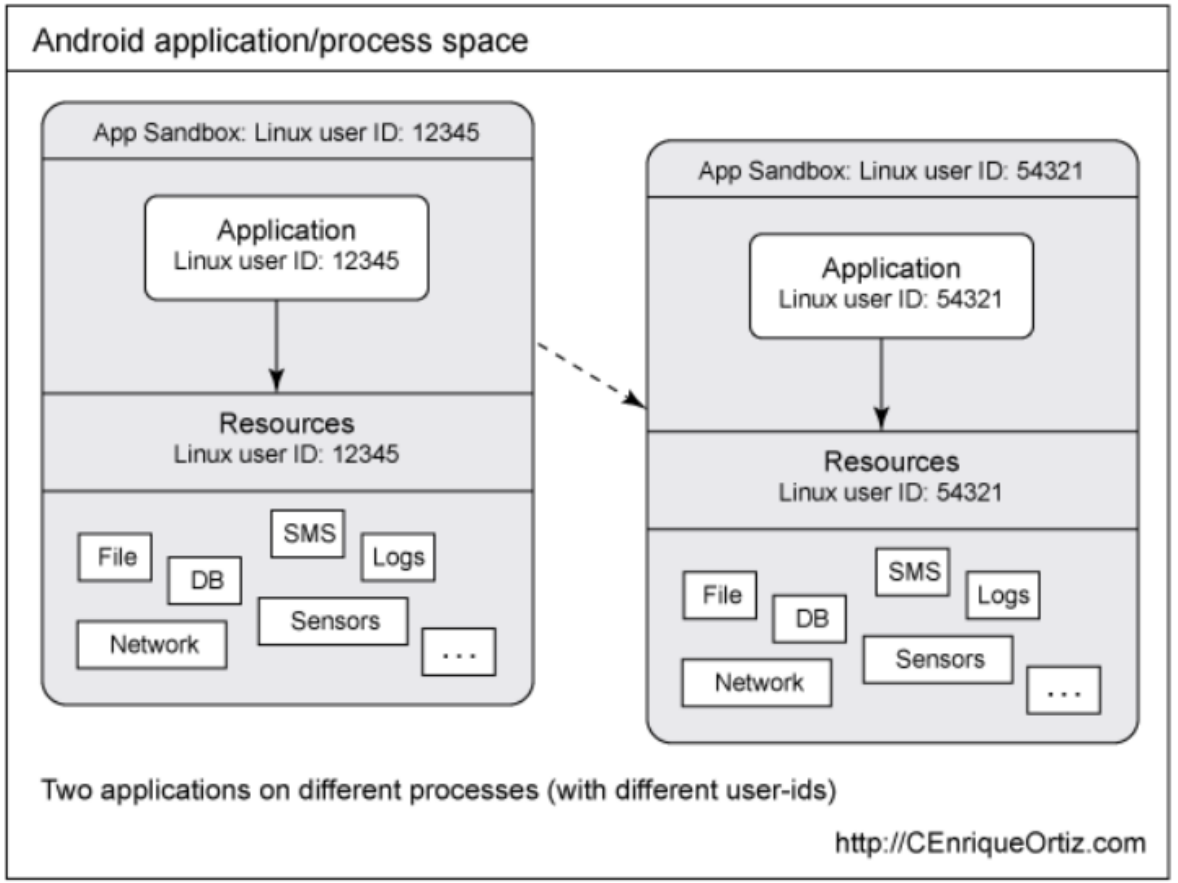
IRM چگونه کار می کند؟

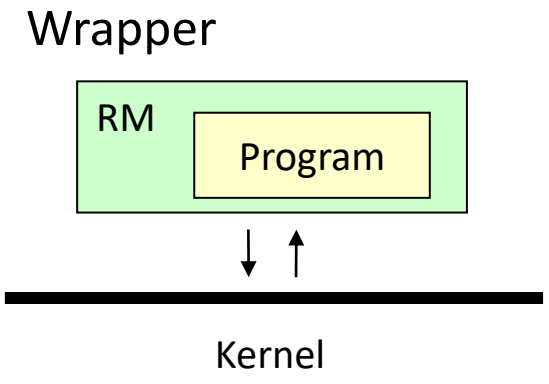


- یک مانیتور مرجع درخطی، یک ساز و کار امنیتی است که در هر عملیات (مرتبط) در یک برنامه مداخله می‌کند و یک سیاست امنیتی از پیش تعریف‌شده را مستقیماً در جریان اجرای برنامه اعمال می‌کند - از این رو "درخطی" است.
- برخلاف مانیتورهای خارجی (مثلاً DAC/MACدر سطح سیستم عامل)، IRMها معمولاً از طریق تجهیز کد موسوم به instrumentation (مثلاً بازنویسی بایت‌کد یا تبدیل کد منبع) در خود برنامه گنجانده می‌شوند.
- بیشتر حکم IPS دارند و جلوی نقض سیاست امنیتی را در سطح خود برنامه می گیرند.
- می توانند مستقلا عمل کنند یا حتی با توجه به امتیاز ریسک بالای IDS بجای اعلام خطر، واقع در سطح برنامه کنشی را سبب شوند(ارتباط با IDS).

بازنویسی کد در IRM







Android sandbox

نمونه ای از لفاف بندی برنامه ها برای اعمال ساز و کارهای امنیتی و شکل دهی یک رصدگر مرجع

افق پیاده سازی ناظر در گزارش اولیه

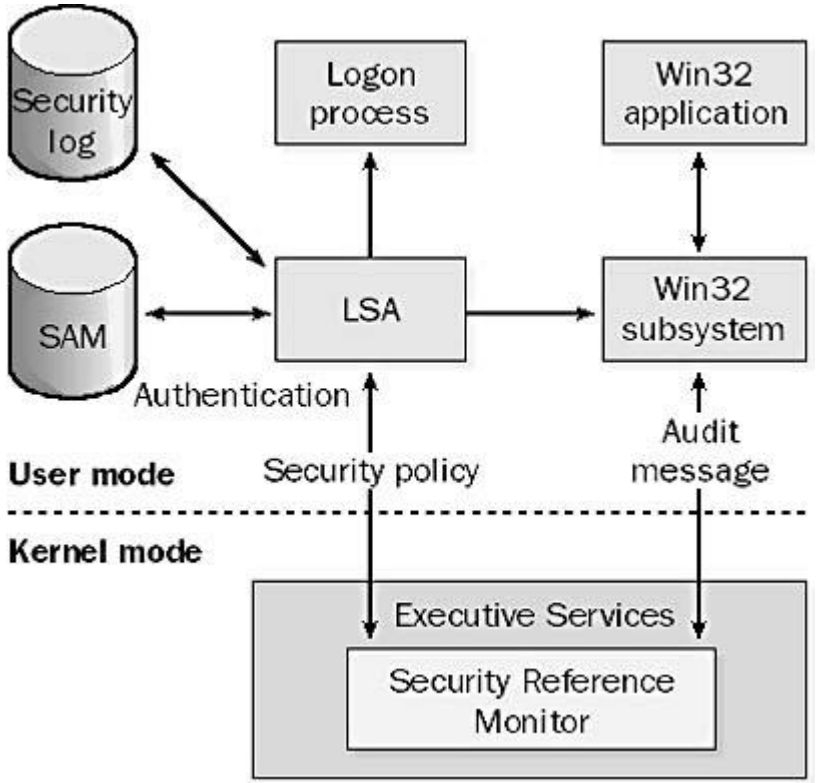
While the design of the reference validation mechanism will not result in an operating system per se, it should help provide a foundation upon which a secure operating system can be developed .

ناظر مرجع ویندوز

- اولین ویندوز هشت سال بعد توسعه داده شد و اولین ناظر امنیتی در آن بعد از یک دهه از توسعه مطابق این مفاهیم به ویندوز اضافه شد.
- ناظر مرجع امنیت ویندوز (SRM) یک جزء در حالت هسته است که مسئول اجرای سیاست‌های کنترل دسترسی در ویندوز است.
- تعدادی از توابع برنامه اجرایی ویندوز (NTOSKRNLL.exe) که با Seشروع می شوند.
- برخی از آنها مستند شده اند و برای استفاده در اختیار کاربر نیز هستند.
- هخوانی همه دسترسی ها را با سیاست ها امنیتی ویندوز چک می کند
- سیستم عامل یکپارچه (monolithic) شرایط سخت تری برای پیاده سازی این مفهوم دارد. چرا؟
- امنیت آن فرمال قابل اثبات نبود و در طول زمان معلوم شد دور می خورد.

در ویندوز SRM (security reference monitor)

1.User logs in (e.g., via Ctrl+Alt+Del).
2.Winlogon invokes the LSA.
3.LSA calls SAM (for local account) or Kerberos/Netlogon (for domain).
4.SAM verifies the password hash → tells LSA if valid.
5.LSA creates an access token containing:
   1. User SID
   2. Group SIDs
   3. Privileges
   4. Integrity level
6.User process runs with this token.
7.When the user tries to open a file:
   1. The SRM (via the I/O Manager) checks if the token has permission to access the file's DACL.
   2. SRM either grants or denies access.



- LSA:Local security authority
- SAM:Security Accounts Manager

از ایده انتزاعیِ اندرسون تا اثبات صوری

- جیمز اندرسون مفهوم *Reference Monitor* را مطرح کرد. اما این مفهوم تا زمانی که ریاضیاتی نشود، صرفاً یک ایده‌ی معماری است. پژوهشگران بلافاصله پس از گزارش ۱۹۷۲ پرسیدند چگونه می‌توانیم ثابت کنیم که یک Security Kernel از نظر ریاضیاتی درست کار می‌کند؟
- اینجا بود که دو اتفاق علمی بزرگ رخ داد:

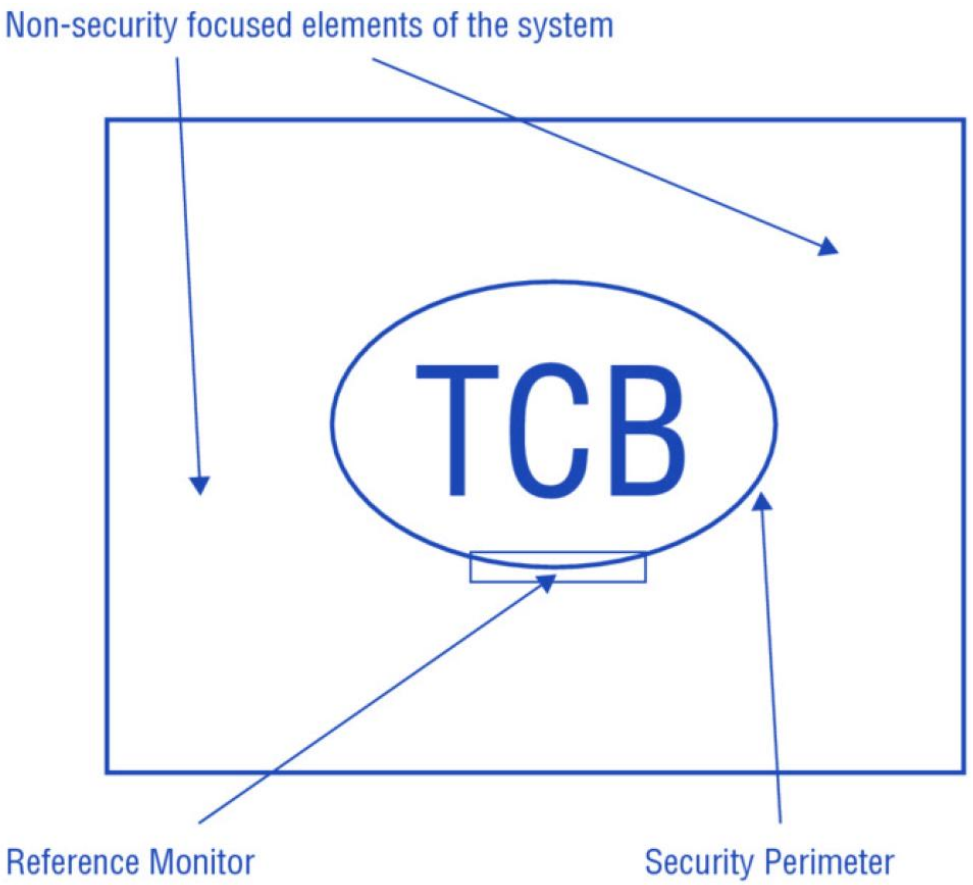
1. مدل‌سازی سیاست امنیتی (Security Policy Modeling):دیوید بل و ریچارد لاپادولا به سفارش وزارت دفاع آمریکا مدلی ریاضی بر پایه نظریه مجموعه‌ها توسعه دادند که به مدل Bell-LaPadula (BLP) معروف شد. این مدل، حالت‌های امن و قوانین دسترسی را فرمالیزه کرد تا ناظر مرجع دقیقاً بداند چه چیزی را باید اجازه دهد یا رد کند.
2. **اثبات صوری هسته :** محققانی چون **ریچارد اوهسیک** و دیگران در مؤسسات پژوهشی تلاش کردند اثبات کنند که کدهای یک *Security Kernel* دقیقاً با مدل ریاضیِ بل-لاپادولا مطابقت دارد. اولین سیستم‌عامل‌هایی که به صورت آکادمیک به عنوان پیاده‌سازیِ واقعیِ ناظر مرجع ساخته شدند، پروژه‌هایی نظیر **Multics** (که اندرسون الهام‌بخش آن بود) و بعدها سیستم‌عامل‌های مینی‌مالی مانند **KSOS** و **PSOS** بودند.

گام دوم: بن‌بستِ مهندسی و تولد مفهومی به نام TCB

- قرار نبود ناظر مرجع به‌تنهایی همه امنیت سیستم‌عامل را انجام دهد؛ قرار بود *تصمیم‌های دسترسیِ مربوط به حفاظت* را به‌طور کامل و قابل‌اعتماد میانجی‌گری کند.
- وقتی در دهه‌ی ۱۹۸۰ وزارت دفاع آمریکا (DoD) خواست این ایده را به عنوان استاندارد رسمیِ خرید سامانه‌های نظامی تدوین کند (که شد همان کتاب نارنجی یا TCSEC)، با یک بن‌بست خنده‌دار و ترسناک روبه‌رو شد: هیچ سیستم‌عامل واقعی، جامع و کاربردی‌ای وجود نداشت که آن هسته امنیتیِ کوچک اندرسون را داشته باشد!
- اگر وزارت دفاع می‌خواست سخت‌گیرانه بگوید فقط سیستم‌هایی امن هستند که مانیتور ارجاع کوچک و اثبات‌پذیرِ خالص داشته باشند، عملاً هیچ کامپیوتری در دنیا تاییدیه امنیتی نمی‌گرفت، چون سیستم‌عامل‌های واقعی (مثل یونیکس‌های اولیه یا سیستم‌های بزرگ مین‌فریم) بزرگ، درهم‌پیچیده و پر از خطاهای پنهان بودند.
- برخلاف تصور اولیه و در واقعیت، اجرای درست سیاست امنیتی به اجزایی فراتر از خودِ منطقِ کنترل دسترسی وابسته است: مثلاً سخت‌افزار، سیستم‌عامل و سازوکارهای دیگری که ممکن است بتوانند کنترل‌ها را دور بزنند یا تضعیف کنند.
- **تفکیک مطلق هسته امنیتی از بقیه سیستم‌عامل از نظر مهندسی و کارایی عملاً شکست خورد یا بسیار پرهزینه به نظر می رسید.**
- اینجا بود که نویسندگان کتاب نارنجی دست به یک تعمیم یا بهتر بگوییم یک عقب‌نشینیِ هوشمندانه زدند:
- اگر نمی‌توانیم کل سیستم را به یک هسته کوچک و معصوم تبدیل کنیم، بیایید دایره را بزرگ‌تر کنیم!
- آن‌ها واژه TCB(مبنای رایانش معتمد) را ابداع کردند و تعریف آن را طوری تعمیم (کش!) دادند که هر چیزی که به نوعی در امنیت سیستم نقش دارد را در بر بگیرد؛ از درایورها و کدهای کمکی بگیرید تا وصله های امنیتی، تنظیمات حافظه و حتی مکانیزم‌های سخت‌افزاری.

پایه رایانشی معتمد (TCB)

- *Trusted Computing Base*
- ترکیبی از سخت‌افزار، نرم‌افزار و کنترل‌هایی که با هم کار می‌کنند تا یک پایگاه قابل اعتماد برای اجرای سیاست امنیتی تشکیل دهند.
  - زیرمجموعه‌ای از یک سیستم اطلاعاتی کامل است. این پایگاه باید تا حد امکان کوچک باشد تا یک تحلیل دقیق بتواند به طور منطقی تضمین کند که سیستم مشخصات و الزامات طراحی را برآورده می‌کند.
  - TCBباید روش‌هایی را برای دسترسی به منابع هم در داخل و هم در خارج از خود TCBارائه دهد.
- بخشی از TCBبرای اعمال کنترل‌های دسترسی بر دارایی‌ها و منابع سیستم (که گاهی اوقات به عنوان اشیاء شناخته می‌شوند) مانیتور مرجع نامیده می‌شود
  - مانیتور مرجع بین هر subject و object قرار دارد و تأیید می‌کند که اعتبارنامه‌های یک موضوع درخواست‌کننده، الزامات دسترسی شیء را قبل از ادامه هرگونه درخواستی برآورده می‌کند.



رویکردهای مهار TCB بزرگ و پیچیده

- **معماری حلقه (Protection Rings)و جداسازی سخت‌افزاری**: برای اینکه نرم‌افزارها و بخش‌های مختلف سیستم‌عامل نتوانند به حریم یکدیگر تجاوز کنند، امنیت به سخت‌افزار تکیه کرد.
- **معماری ریزهسته (Microkernel Architecture – نسل اول)**:در دهه ۱۹۸۰ و ۱۹۹۰ (مثل پروژه **Mach**)، تلاش شد تا حد امکان اجزای TCB از داخل هسته (Kernel Space) بیرون کشیده شده و به صورت سرویس‌های فضای کاربر (User Space) اجرا شوند. مثلا اگر درایور کارت شبکه یا فایل‌سیستم کرش کند یا آسیب‌پذیری داشته باشد، کل TCB سقوط نکند و فقط همان سرویس ری‌استارت شود.
- **۳. ایزوله‌سازی و مجازیسازی (Virtualization & Hypervisors)**: وقتی نتوانستند TCB یک سیستم‌عامل ۲۰۰ میلیون خطی را کوچک یا ایمن کنند، یک لایه **پایین‌تر** از سیستم‌عامل اضافه کردند مثل Xenیا VMware ESXi
- **سخت‌افزارسازیِ ریشه اعتماد از TPMتا ... ,SGX**
- **کاهش نرم افزاری آسیب پذیری بجای اعتماد مطلق!**

میکروکرنل

«یک کرنل فقط زمانی خوب است که به شما اجازه دهد کارهای احمقانه انجام ندهید.»

ریچارد رشید (Richard Rashid)، یکی از اصلی‌ترین طراحان میکروکرنل Mach در دانشگاه کارنگی ملون.

داستان Minix 3 و تست مرگبار:
تزریق عمدی باگ به درایورها

- سیستم‌عامل میکروکرنلی معروفی که **اندرو تاننبام** آن را ساخت و الهام‌بخش اولیه لینوکس توروالدز برای نوشتن لینوکس بود. در نسخه **Minix 3**، هدف تاننبام ساخت سیستمی بود که ادعا می‌کرد: «**کرش کردن یا هک شدن درایورها هرگز نباید سیستم را زمین بزند.**»
- تیم دانشگاه Vrije آمستردام برای اثبات امنیت و پایداری این میکروکرنل سنتی، دست به آزمایشی جذاب زدند. آن‌ها کدی به نام **Software Fault Injector** نوشتند. این ابزار به صورت رندوم، کدهای مخرب، خطاهای سرریز حافظه، حلقه‌های بی‌نهایت و اشاره‌گرهای نامعتبر را داخل کد درایورهای کارت شبکه، دیسک و پردازش صدا تزریق می‌کرد (دقیقاً رفتاری مشابه اکسپلویت‌های مهاجمین یا دَرایورهای پر از باگ). این کار را هزاران بار تکرار کردند.
- در لینوکس یا ویندوز، اگر یک درایور کارت شبکه یا هارد دیسک دچار سرریز بافر شود، کل سیستم با خطای صفحه آبی خاموش می‌شود.
- اما در Minix 3 درایورها برنامه‌هایی کاملاً عادی در فضای کاربر (User Space) بودند.
- سرویسی به نام **Reincarnation Server** (سرور تناسخ!) وجود داشت. به محض اینکه یک درایور بر اثر تزریق باگ هک یا کرش می‌کرد، میکروکرنل جلوی نفوذ به حافظه بقیه را می‌گرفت، سرویس تناسخ درایور معیوب را فوراً می‌کشت، حافظه‌اش را پاک می‌کرد و یک نسخه تر و تمیز از آن را در کسری از میلی‌ثانیه بالا می‌آورد؛ بدون اینکه کاربر یا سیستم حتی متوجه شوند!
- آیا مینیکس زنده است و شما با آن سر و کار دارید؟

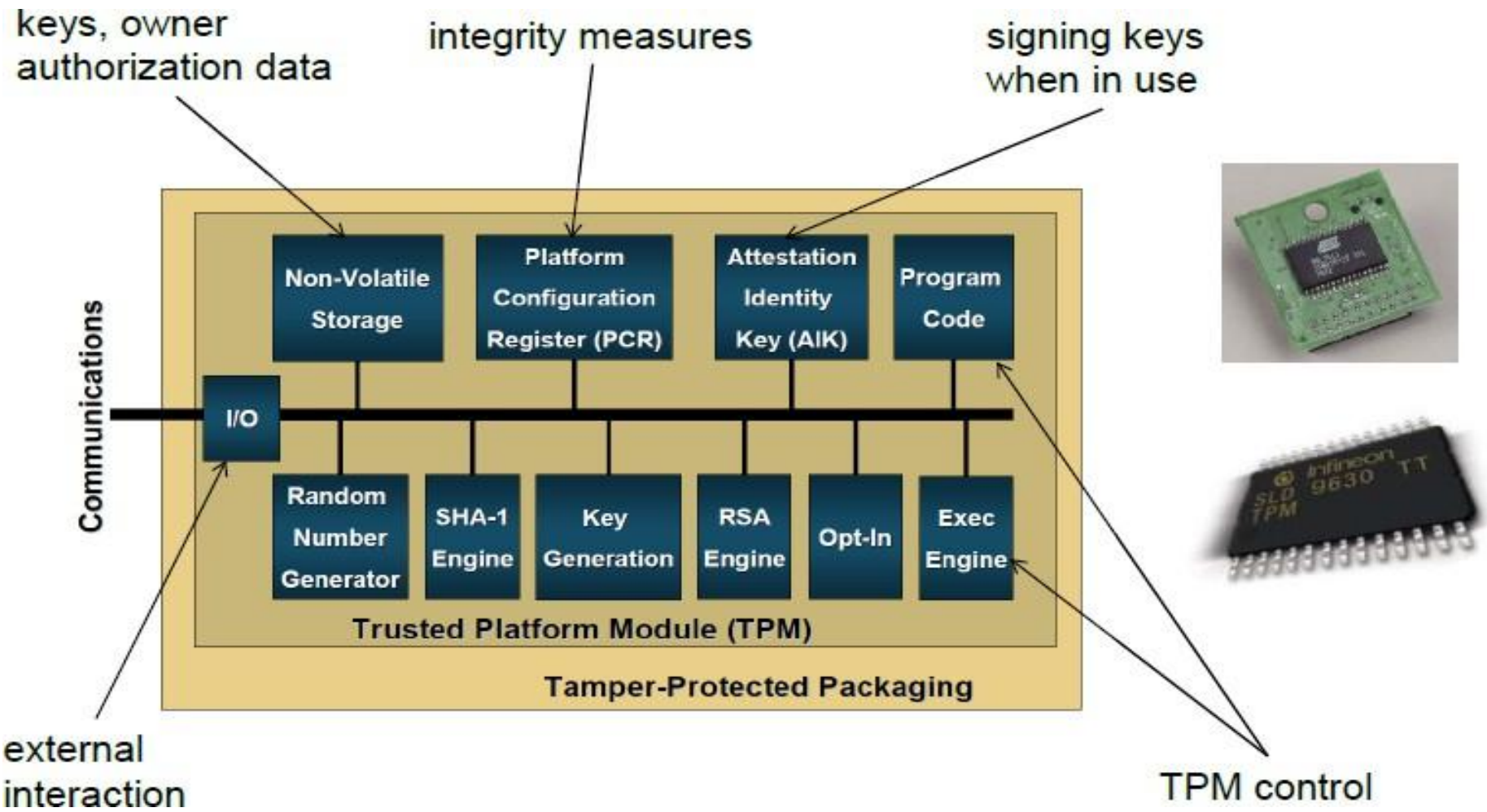
جنگ تئوریک: تاننبام در برابر لینوس توروالدز (میکروکرنل در برابر یکپارچه)

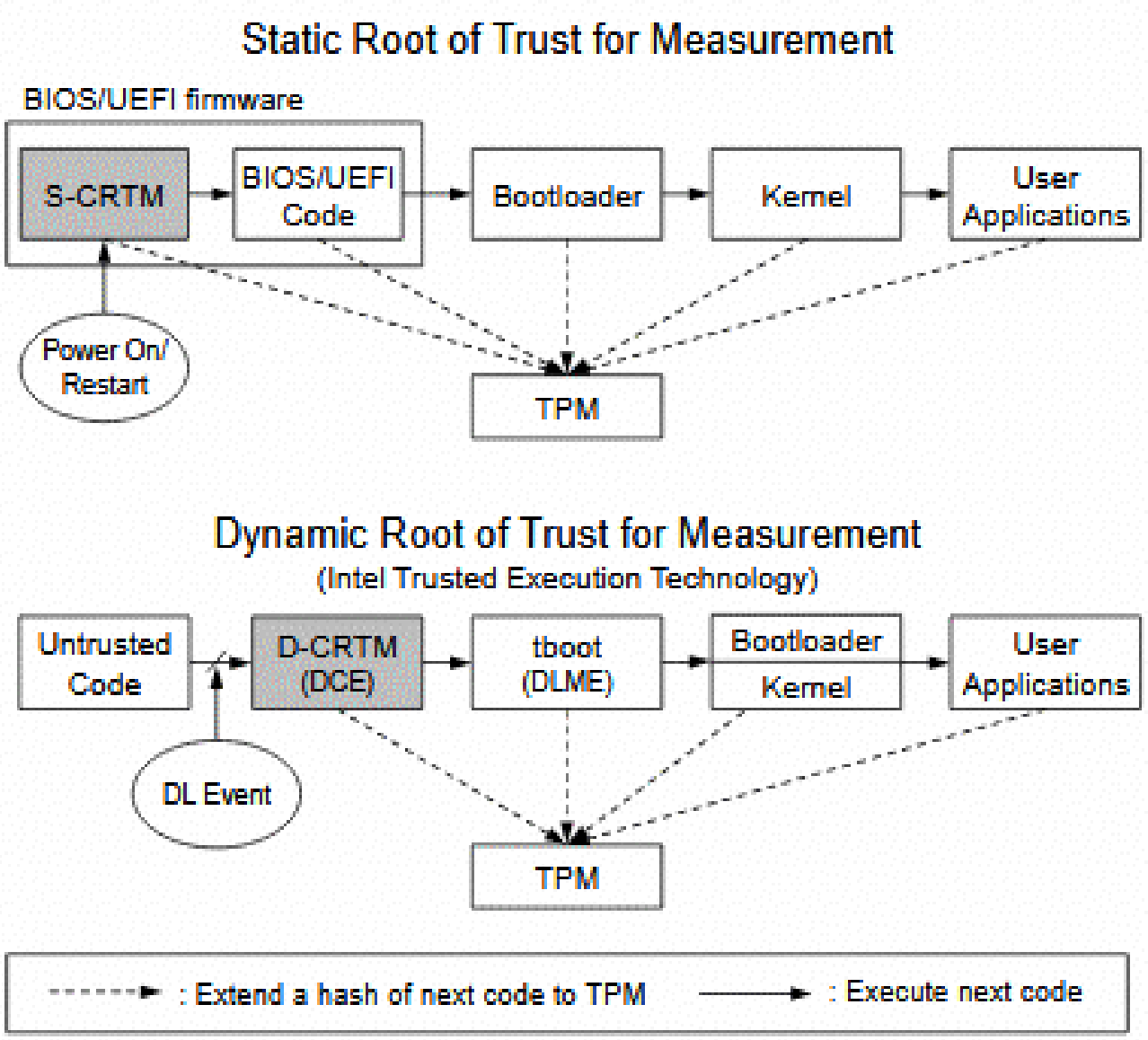
- در سال ۱۹۹۲، بحث بسیار تند و تاریخی بین اندرو تاننبام (پدر میکروکرنل MINIX) و لینوس توروالدز (خالق لینوکس) در یوزنت شکل گرفت که به یکی از کلاسیک‌ترین نقل‌قول‌های تاریخ کامپیوتر تبدیل شد.
- تاننبام: «من هنوز سر حرفم هستم که نوشتن یک هسته یکپارچه در سال ۱۹۹۱ یک گام به جلو به سمت دهه ۱۹۷۰ است. [...] ارتقا دادن لینوکس به یک سیستم‌عامل مدرن مثل این است که یک موتور دوچرخه را روی یک بوئینگ ۷۴۷ بگذارید!
- لینوس توروالدز (در پاسخ به تاننبام): «تئوری عالی است، ولی در عمل شکست می‌خورد... تاننبام ممکن است سیستم‌عامل مینیکس خودش را داشته باشد، اما این لینوکس است که دنیا را تسخیر می‌کند، دقیقاً به این دلیل که معماری کاربردی‌تری دارد.
- به نظر شما حق با چه کسی بود؟

تکلیف شما

- درباره معماری هیبریدی (میکرو و غیرمیکروکرنلی) قصه های تاریخی MACH را بخوانید

TPM چیست و چگونه کار می کند





Application	Static Root of Trust (SRoT)	Dynamic Root of Trust (DRoT)	Companies/Products
System Boot	☑	✗	Microsoft, Red Hat, Canonical
Secure Updates	✗	☑	Microsoft, Apple, Google
Key Storage	☑	✗	Intel, AMD, Infineon
Temporary Processing	✗	☑	HSBC, Chase, Bank of America
Fixed Firmware	☑	✗	Cisco, Juniper, Huawei
Runtime Isolation	✗	☑	Riot Games, Epic Games, Valve
Secure Hardware	☑	✗	Gemalto, Thales, Visa
Cloud Processing	✗	☑	Microsoft Azure, AWS, Google Cloud



Their definition of `security' is controversial; machines built according to their specification will be more trustworthy from the point of view of software vendors and the content industry, but will be less trustworthy from the point of view of their owners.

In effect, the TCG specification will transfer the ultimate control of your PC from you to whoever wrote the software it happens to be running. (Yes, even more so than at present.)



There's also a lot I don't like, and am scared of. My fear is that Pd will lead us down a road where our computers are no longer our computers, but are instead owned by a variety of factions and companies all looking for a piece of our wallet.

رایانش معتمد یا خیانت آمیز؟!



- In the past, these were isolated incidents. “Trusted computing” would make the practice pervasive. “Treacherous computing” is a more appropriate name, because the plan is designed to make sure your computer will systematically disobey you.
- In fact, it is designed to stop your computer from functioning as a general-purpose computer. Every operation may require explicit permission.
- The technical idea underlying treacherous computing is that the computer includes a digital encryption and signature device, and the keys are kept secret from you.
- Proprietary programs will use this device to control which other programs you can run, which documents or data you can access, and what programs you can pass them to.
- These programs will continually download new authorization rules through the Internet, and impose those rules automatically on your work. If you don't allow your computer to obtain the new rules periodically from the Internet, some capabilities will automatically cease to function.

تتراگون، ناظری سوار بر ebpf

- درباره تتراگون و اینکه چقدر در عمل به ناظر مرجع نزدیک شده است تحقیق کنید.

«تتراگون داده‌های غنی‌ای در اختیار تیم امنیتی ما می‌گذارد که شبکه، فرآیندها (Process) و متادیتای کوبرنتیز را به یک رکورد رخداد واحد متصل می‌کند. داشتن این تصویر ترکیبی به ما اجازه می‌دهد رفتارهای شبکه را تا سطح نود، نیم‌اسپیس، پاد و کانتینر زیر ذره‌بین ببریم. علاوه‌بر این، راه‌اندازی‌اش سریع بود و سربار (Overhead) بسیار پایینی داشت که این برای مقیاس کاری ما حیاتی است.

جیسون ستینا (Jason Cetina) – مهندس ارشد امنیت در گیت‌هاب (GitHub)

اکثر سیستم‌های امنیتی، تماشاگرانی هستند که بعد از دزدیده شدن داده‌ها فقط یک نوتیفیکیشن می‌فرستند. ارزش واقعی تتراگون در این است که فقط یک ابزار ثبت لاگ نیست؛ تتراگون به لطف قلاب‌های امنیتی هسته، یک "سامانهٔ اعمال سیاست درجا" (Runtime Enforcement) است؛ یعنی می‌تواند پیش از آنکه فرآیند مخرب گامی به جلو بردارد، آن را در همان سطح کرنل متوقف کند.

توماس گراف (Thomas Graf) – هم‌بنیان‌گذار Cilium و نویسنده کرنل لینوکس

نقد طنز آلود TCB

The irony of the TCB concept was that it was supposed to be a small, pristine security core that we trust to keep the rest of the system secure; but in practice, as designers saw that no real operating system could function without messy code, bug-ridden drivers, and complex features, they were forced to stretch and expand the definition of the TCB until it effectively encompassed the entire multi-million-line, flaw-ridden operating system.

Rather than securing the system, they widened the circle of trust to include every dubious and unsecure piece of code just to pass the standard on paper—a bitter irony where the term “Trusted Computing Base” became synonymous with “whatever we don’t have the patience to secure but have no choice except to trust.”

Lipner, Steven B. "The birth and death of the orange book." *IEEE Annals of the History of Computing* 37, no. 2 (2015): 19-31.

گام سوم: بازگشت به مفهوم اصیل ناظر مرجع

در گام سوم، مهندسان امنیت دیگر مجبور نیستند بین امنیتِ محضِ کوچک و کاراییِ سیستم‌های بزرگ یکی را انتخاب کنند. فناوری‌های جدید به ما اجازه داده‌اند که اصول سه‌گانهٔ اندرسون (غیرقابل‌دستکاری، همیشه فعال، قابل‌راستی‌آزمایی) را در مقیاس‌های پیچیده پیاده کنیم.

اثبات صوری قلاب ها در لینوکس

تتراگون را می‌توان تجسمِ عملیِ یک «ناظر مرجعِ زمان‌اجر در دنیای لینوکس دانست؛ ابزاری که با تزریق بایت‌کد eBPF به اعماق هسته و نشستن روی قلاب‌های بحرانی کرنل (kprobes/tracepoints)، شرط کلاسیک میانجی‌گریِ همیشگیِ دسترسی را بدون نیاز به بازنویسی سنگینِ TCB احیا می‌کند و با بهره‌گیری از قابلیت مسدودسازی بلادرنگ و دستکاری مقادیر بازگشتی سیستم‌کال‌ها، از دورزدنِ حفاظتی جلوگیری کرده و عملاً نقش دروازه‌بان و کنترل‌گرِ غیرقابل‌انعطاف دسترسی‌ها را در سطحِ کرنل ایفا می‌کند.

کنار گذاشتن TCB بزرگ در اعتماد صفر

Zero Trust می‌گوید فرضِ امنیتِ پیرامونی و اعتماد خودکار به همه‌ی اجزای داخلیِ TCB یک خطای راهبردی بوده است. در معماری هیچ اعتماد بر اساس استاندارد (NIST SP 800-207)، مفهوم TCB سنتیِ همه‌چیز-متمرکز عملاً کنار گذاشته می‌شود یا به شدت کوچک می‌شود.

ایده این است: به هیچ موجودیتی (کاربر، دیوایس، سرویس درون شبکه) به صرف اینکه داخل شبکه قرار دارد اعتماد نکن. بارِ امنیت از روی کلِ TCB میزبان برداشته می‌شود و روی دو جزءِ بسیار متمرکز می‌افتد:

PDP (Policy Decision Point) مغز متفکر و سیاست‌گذار (که تصمیم می‌گیرد چه کسی اجازه دسترسی دارد).

PEP (Policy Enforcement Point) دروازه‌بانِ سبکی که درخواست‌ها را رهگیری کرده و با استعلام از PDP، دسترسی را لحظه‌ای باز یا مسدود می‌کند.

بازگشت به اثبات صوری:

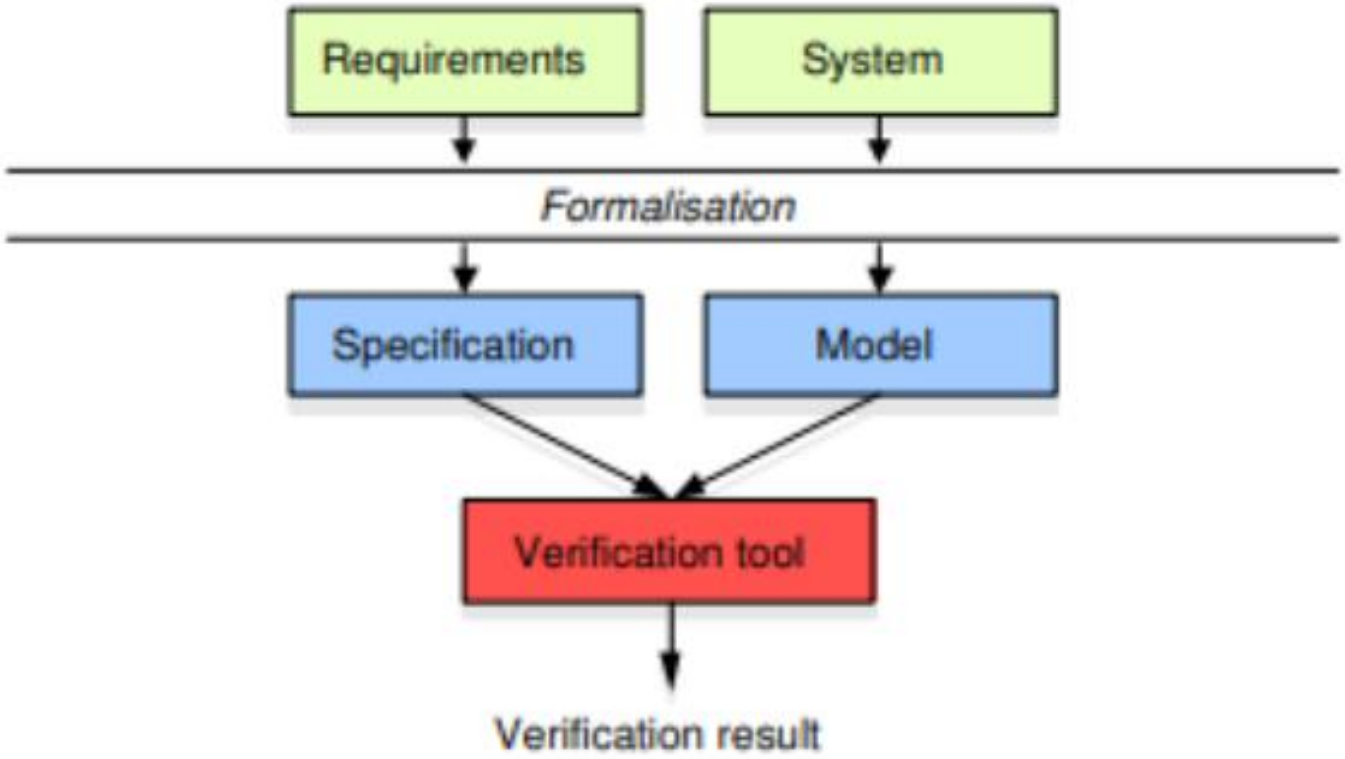
اندرسون می‌گفت ناظر مرجع باید اثبات‌پذیر باشد. در گام دوم (کتاب نارنجی)، اثبات‌ها دستی و بسیار محدود بودند. اما در گام سوم، با پیشرفت ابزارهای خودکارِ ریاضیاتی، پژوهشگران توانستند ثابت کنند که کدِ یک ریزهسته seL4 از نظر منطقی دقیقاً مطابق با مدل امنیتی‌اش است و هیچ باگ یا حفره‌ای در آن وجود ندارد. این یعنی هسته سیستم‌عامل دیگر فرضاً امن نیست؛ بلکه از نظر ریاضی اثبات شده که امن است.

SEL4

- seL4یک میکروکرنل متن‌باز با کارایی بالا است که برای سیستم‌های ایمنی و امنیتی حیاتی طراحی شده است. ویژگی بارز آن این است که به طور رسمی تأیید شده است.
- ویژگی‌های کلیدی:
  - نوشته شده با زبان Cو مقداری اسمبلی.
  - طراحی مینیمال (حدود ۱۰ هزار خط کد)، با رعایت اصل حداقل امتیاز.
  - ایزولاسیون از طریق قابلیت‌ها (ارجاعات شیء امن) اعمال می‌شود.
- seL4اولین هسته سیستم عامل همه منظوره با اثبات بررسی شده توسط ماشین است:
  - صحت عملکردی: پیاده‌سازی با مشخصات رسمی آن مطابقت دارد.
  - ویژگی‌های امنیتی: یکپارچگی و محرمانگی را (از طریق کنترل جریان اطلاعات) اعمال می‌کند.
  - اعمال کنترل دسترسی: هسته تمام دسترسی‌ها به منابع را طبق قابلیت‌ها انجام می‌دهد و به عنوان یک ناظر مرجع عمل می‌کند.

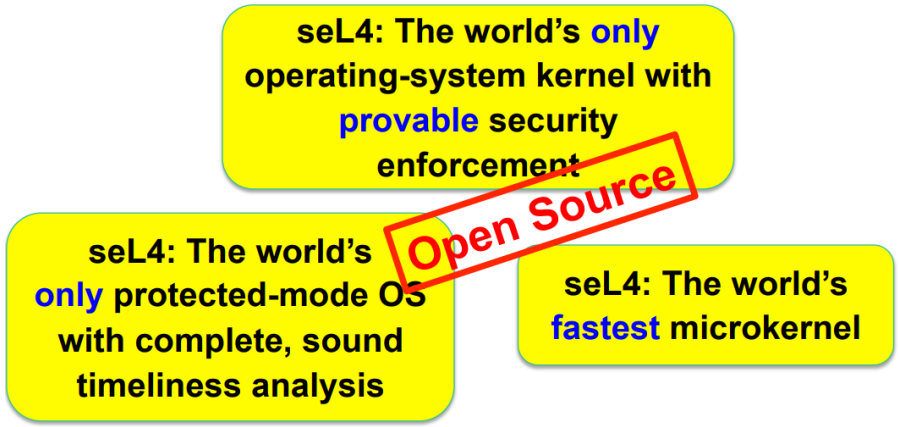
اثبات صوری

- مشخصات انتزاعی (امنیت سطح بالا + مدل عملکردی)
  - نوشته شده در Isabelle/HOL
- مشخصات اجرایی (به زبانی شبیه Haskell)
  - اصلاح مشخصات انتزاعی را اثبات کرد
- پیاده‌سازی C
  - ترجمه شده به معانی Isabelle/HOL
  - صحت عملکردی را اثبات کرد: کد C با مشخصات اجرایی مطابقت دارد
- باینری (ARM)
  - بعداً (از طریق seL4/CAmkES) تأیید شد که کامپایل، معانی را حفظ می‌کند.
- ☑ این کار به بیش از ۲۰۰۰۰۰ خط اسکریپت اثبات نیاز داشت – همه به صورت دستی هدایت می‌شدند اما توسط ماشین بررسی می‌شدند.



seL4 ناظر مرجع است

- همیشه فراخوانی شده → seL4تمام دسترسی‌ها به سخت‌افزار و سایر فرآیندها را از طریق فراخوانی‌های کنترل‌شده هسته میانجی‌گری می‌کند.
- ضد دستکاری → مدل امنیتی مبتنی بر قابلیت seL4تضمین می‌کند که فقط افراد مجاز می‌توانند به اشیاء دسترسی داشته باشند؛ به‌علاوه، با تأیید رسمی، نمی‌توان آن را با نقص‌های پیاده‌سازی مختل کرد.
- قابل تأیید → seL4به‌طور رسمی تأیید شده است و سومین الزام را به‌طور دقیق برآورده می‌کند.
- بنابراین، seL4خود یک مانیتور مرجع ایده‌آل را در سطح هسته پیاده‌سازی می‌کند.



این تأیید رسمی به این معنی است:

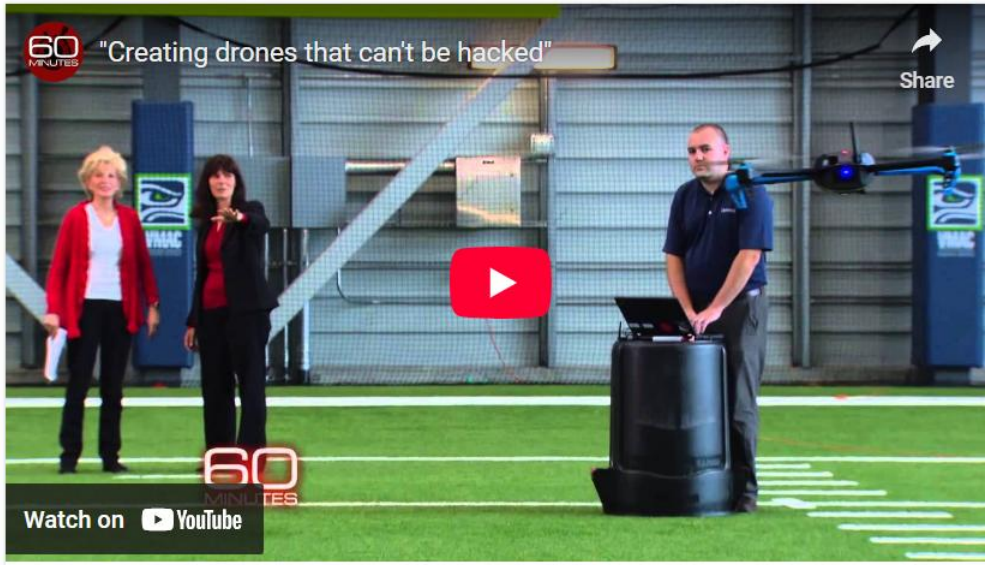
- ✓ هسته seL4از نظر ریاضی تضمین می‌کند که دقیقاً مطابق مشخصات رفتار می‌کند.
- ✓ هیچ اشکالی (مانند سرریز بافر، شرایط رقابتی) وجود ندارد که بتواند مدل امنیتی آن را نقض کند.
- ✓ TCBبه حداقل رسیده و قابل اعتماد است.

ماجرای هک و آزمون DARPA: پروژه HACMS

- در حدود سال‌های ۲۰۱۵–۲۰۱۷، آژانس پروژه‌های پژوهشی پیشرفته دفاعی آمریکا (DARPA) پروژه‌ای به نام **HACMS** (سیستم‌های سایبری-فیزیکی مقاوم در برابر نفوذ) اجرا کرد.
- هدف: یک هلیکوپتر/پهپاد بدون سرنشین واقعی بوئینگ به نام **Uncrewed Little Bird** (نسخه خودمختار هلیکوپتر معروف AH-۶).
- **سناریوی آزمایش:**

1. یک تیم زبده از هکرهای کلاه‌سفید (تیم قرمز) استخدام شدند.
2. به آن‌ها **کد منبع باز** هلیکوپتر، دیاگرام‌های سخت‌افزاری، و حتی **دسترسی فیزیکی مستقیم** به یکی از کامپیوترهای جانبی پرواز (سیستم مأموریت و دوربین) داده شد.
3. در حالت عادی، سیستم‌های پهپاد اگر روی هسته‌های سنتی اجرا می‌شدند، نفوذ به بخش دوربین به معنای نفوذ به باس CAN هلیکوپتر و سپس گرفتن کنترل پرواز (ملخ‌ها، سکان و موتور) بود.

- **معماری جدید بر پایه sel4:**
- محققان معماری را بازنویسی کردند. لایه‌های ناوبری، کنترل پرواز، دوربین‌ها و رادیو درون کپسول‌های ایزوله روی میکروکرنل sel4 قرار گرفتند. هر کپسول فقط و فقط با مجوزهای ریاضیاتی سخت‌گیرانه حق ارسال پیام به کپسول دیگر را داشت.
- **نتیجه چه شد؟**
- تیم قرمز هفته‌ها فرصت داشت تا از طریق کامپیوتر دوربین به کامپیوتر اصلی پرواز نفوذ کند.
- آن‌ها توانستند بخش دوربین و چند سرویس غیرحیاتی را کامپلاً اکسپلویت و کرش کنند؛ **اما هرگز نتوانستند از جعبه شنی خود خارج شوند.**
- خلبان آزمایشی و کامپیوتر ناوبری هلیکوپتر بدون کوچک‌ترین لرزش یا نقصی به پرواز ادامه دادند.
- ارزیاب ارشد DARPA اعلام کرد: *«این پهپاد احتمالاً هک‌ناپذیرترین وسیلهٔ نقلیهٔ هوایی است که تاکنون ساخته شده است.»*



DARPA HACMS EMPLOYED FORMAL VERIFICATION METHODS TO DEVELOP HACK PROOF UNMANNED VEHICLES , WEAPONS , SATELLITES, AND COMMAND AND CONTROL SYSTEMS

Rajesh Uppal · February 17, 2022 · AI & IT, Cyber & IW · Comments Off · 1,366 Views

Cheri

- CHERIپردازنده‌های سنتی را با موارد زیر تقویت می‌کند:
- قابلیت‌های سخت‌افزاری: اشاره‌گرهای غنی از مجوز و غیرقابل جعل که شامل محدوده‌ها، مجوزها (خواندن/نوشتن/اجرا) و هویت شیء می‌شوند.
- محافظت دقیق از حافظه: از سرریز بافر، استفاده پس از آزادسازی و سایر اشکالات تخریب حافظه در سطح سخت‌افزار جلوگیری می‌کند.
- این امر بخشی از پایگاه محاسباتی قابل اعتماد TCBرا از نرم‌افزار به سخت‌افزار منتقل می‌کند و ساخت سیستم‌های امن را آسان‌تر می‌سازد.

رویکردهای نو ناظر در هسته سیستم عامل و سخت افزار

Project	Tamperproof	Always Invoked	Verifiable	Notes
seL4	☑	☑	☑	Best-known verified kernel
CHERI	☑	☑	◇ (ISA only)	HW-assisted memory safety
MI6	☑	☑	◇ (partially)	Secure multicore enclaves
Ironclad	☑	☑	☑	Verified apps + OS stack
Genode	◇ (modular)	☑	◇	Capability-based

seL4 + CHERI = Stronger Reference Monitor

تیم seL4 در Data61 CSIROو همکارانش seL4را برای اجرا روی سخت‌افزارهای دارای قابلیت CHERI مثلاً CHERI-RISC-Vپورت کرده‌اند.محققان در حال اثبات فرمال امنیت این ترکیب هستند.

خواندنی(۲ مورد)

- Sá, Bruno, Donato Ferraro, Everton de Matos, Rafail Psiakis, Andrea Marongiu, Andrea Bastoni, and Sandro Pinto. "Advanced Security through Hardware Capabilities: A Comprehensive Survey on CHERI Technology." *ACM Computing Surveys* 58, no. 15 (2026): 1-36.
- https://sel4.systems/use.html